Freight Rate Prediction: Approach and Validation

Ameema Rashid · Spotter AI, Machine Learning Engineer assessment · November 2026

1. Summary

I built a model that predicts the posted rate of a freight load from its distance, weight, pickup and delivery locations, equipment type and day of the week. The 12,000 loads to predict (1 November to 31 December 2025) all come after the labelled data (1 January to 31 October 2025), so I validated **only with time-based splits**: modelling choices were made on four walk-forward months (May to August), and September to October was kept as a protected two-month checkpoint. That checkpoint changed one decision: it rejected a model using market_index (section 4), and a replacement I had specified in advance was used instead.

The submitted model is a gradient-boosting regressor (scikit-learn's HistGradientBoostingRegressor) trained on log(rate per mile). On September to October it scored **$98.57 mean absolute error (MAE) and 4.34% MAPE**, against $140.53 and 6.40% for a simple baseline. Spotter's accuracy metric is not stated, so I use MAE in dollars as my main metric and also report MAPE, RMSE and bias.

2. Data and data quality

There are 48,000 labelled loads (Jan to Oct 2025), 12,000 loads to predict (Nov to Dec 2025) and a 31-row December chart file. Distance and equipment explain most of the price: a baseline that only looks up the median rate per mile by equipment and distance band cuts the error of a single overall median by about 88%. I found four data-quality issues:

Issue	Rows (train / valid)	What I did	Why
Negative weights	292 / 145	Flipped the sign	Flipped values match the positive ones in range and per-equipment medians, so they look like sign errors
Missing weight	300 / 165	Filled with the training median	Rows with a missing weight are priced like the rest, so the gaps look random
Missing market_index	374 / 249	Filled with the same-date mean from the same file	Values barely vary within a date (about ±0.03)
Rates 2–6× off similar loads	677 (1.41%) in training	Excluded from training only; every test row is still scored	337 are below half and 340 above twice the median rate per mile for their equipment and distance band, spread evenly over months and equipment: consistent with label errors, not price spikes

Two columns have no documented meaning: market_index and quote_signal. I dropped quote_signal because its link to price flipped between months, it made the August test worse (MAE +$38.44), and the December file has no value for it. market_index is discussed in section 4. All cleaning values (weight median, city coordinates, the price table used to flag label errors) are learned from the training rows only, by one shared function used in both training and prediction.

3. How I split and validated the data

Why time-based. The loads to predict lie entirely after the labelled period, so the model always predicts the future. A fair test must do the same. A random split would let the model learn from months that come after the test rows, and price levels move from month to month (the baseline over-priced January by $134 on average and under-priced June by $112), so a random split mixes those level shifts into the score.

Split	Train on	Test on	Rows	Used for
4 walk-forward folds	1 Jan to the end of the month before the test month	May, Jun, Jul, Aug (one month each)	Test 4,913 / 4,783 / 4,912 / 4,759	Modelling choices
Sep–Oct holdout	1 Jan – 31 Aug	1 Sep – 31 Oct	38,477 train / 9,523 test	Checkpoints only; never for tuning (it rejected the market model)
Final fit	1 Jan – 31 Oct	Nov – Dec loads (no labels)	47,323 train / 12,000 predicted	The submission

- **No leakage:** raw rows are split by date first; cleaning values and the label-error price table are then learned from the training side only. Every model goes through the same pipeline and the same metrics.
- **Why four folds:** one month is noisy, and a fix suggested by August and judged only on August would be circular. Before experimenting, I fixed an adoption rule: a change that adds complexity is kept only if mean MAE improves by at least $1, it wins at least 3 of 4 folds, and mean MAPE rises by no more than 0.10 points.
- **Why Sep–Oct as the holdout:** it has the same two-month horizon as Nov–Dec and falls in the lower market period after late July.
- **Metrics:** MAE in dollars is primary (it is in the deliverable's units and is less distorted by the 1.4% label errors than RMSE); MAPE is second; RMSE and bias (predicted minus true) are also reported.
- **Disclosure:** the Sep–Oct holdout was scored for three models: the market-index model (first look, which failed), its pre-registered replacement (second look), and the submitted model after a settings change (third look, report only; no choice was made from it).

Random vs time split (measured on my Phase 4 tuned model, not the submitted one): a random split gave clean-row MAE of $67.49 to $68.12 over 3 seeds, against $42.31 on the Sep–Oct time split. Here the random split was harsher, because its test rows include the early months with large price-level swings. Either way, the score depends heavily on how the split is drawn, which is why my split copies the real task.

4. Model and results

- **Baseline (B1):** the median rate per mile for each equipment type and distance band, multiplied by distance.
- **Model:** HistGradientBoostingRegressor (scikit-learn 1.9.1). It handles non-linear distance effects and interactions without hand-built curves and is fast. Settings: library defaults (learning rate 0.1, 100 iterations), random_state 42, and early stopping switched off because its internal 10% hold-out is not described as time-ordered.
- **Target:** log(rate per mile), converted back to dollars as exp(prediction) × distance inside the model. Errors become proportional, and log(rate) lost to it on all 4 folds.
- **Features (10):** distance, weight, pickup and delivery latitude/longitude, equipment (one-hot) and weekday. I used coordinates instead of city names, so the 8 cities that appear only in the prediction file still get a sensible input.
- **market_index dropped:** the walk-forward folds favoured it, but on Sep–Oct the market model scored $188.31 MAE, worse than the baseline's $140.53, and under-priced by 5.6%. Its link to price broke after a drop in late July, so a model that relies on it cannot be trusted for Nov–Dec.

Walk-forward MAE (dollars) and MAPE, baseline vs submitted model

Fold	B1 MAE	Submitted MAE	Difference	B1 MAPE	Submitted MAPE
May	165.35	155.73	−9.62	7.66%	6.90%
Jun	212.14	212.18	+0.04 (tie)	8.18%	7.87%
Jul	148.58	119.79	−28.78	6.31%	4.76%
Aug	136.60	90.07	−46.53	6.39%	4.15%
Mean	**165.67**	**144.44**	**−21.22**	**7.14%**	**5.92%**

The submitted model has the lower MAE in 3 of 4 months; June is a tie (4 cents worse on MAE, better on MAPE). MAPE is lower in every month. Differences are computed from unrounded values, so they can differ by 1 cent from the rounded columns.

Sep–Oct holdout (9,523 loads)

Model	MAE	MAPE	Clean-row MAE
B1 baseline	$140.53	6.40%	$85.01
Submitted model (third look, report only)	$98.57	4.34%	$42.53

Clean-row MAE leaves out holdout rows flagged as likely label errors; the headline uses all rows. On clean rows the submitted model under-prices by about 0.77% on average.

Unseen cities (measured on the Phase 4 tuned model): removing 8 training cities and scoring the same 1,226 August loads raised MAE by only $0.56, from $71.63 to $72.20 (the difference is taken before rounding), still far below the baseline's $122.09 on those loads.

5. Stability

The final model is deterministic (changing the random seed changed no prediction), so I measured stability with a paired bootstrap that resamples **test days** (1,000 repeats, seed 42) over the four walk-forward months, scoring every model on the same resampled days. I resampled days rather than loads because loads on the same day share a price level, so treating them as independent overstates certainty.

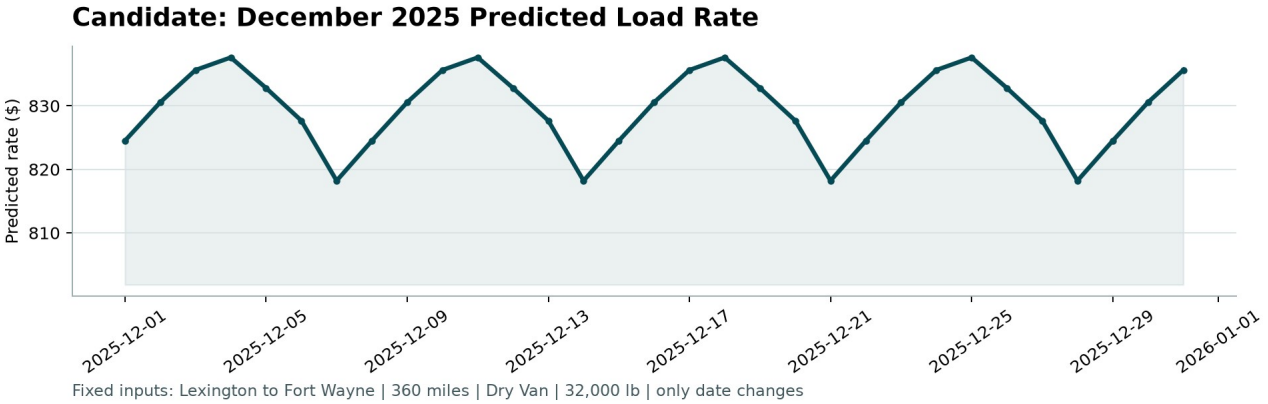
- **Excluding label errors is a robust gain:** keeping them makes the model $7.59 per load worse (95% interval $6.34 to $8.68; worse in 4 of 4 months).
- **Weekday is within noise:** removing it makes the model $1.42 worse, but the interval (−$1.26 to $3.91) includes zero. I kept weekday because it passed my adoption rule.
- **Tuned settings are within noise:** the tuned boosting settings (learning rate 0.05, 400 iterations) had an MAE $0.11 **lower** than the library defaults, but the 95% interval for the difference (tuned minus defaults) runs from −$0.34 to +$0.11 and includes zero. My re-check rule, fixed in advance, keeps the simpler option when it is no more than $1 worse in mean MAE and MAPE rises by no more than 0.10 points, so I switched to the defaults. The switched model is the submitted one: $98.57 MAE / 4.34% MAPE on Sep–Oct (a third, report-only look) against $140.53 / 6.40% for the baseline.

6. Final training and output files

One command, `python src/train_predict.py`, retrains the submitted model on all 48,000 labelled loads (1 January to 31 October 2025), leaving out the 677 likely label errors (47,323 training rows), and writes both output files. `validation_predictions.csv` holds 12,000 predictions in the template's order, from $184.61 to $6,821.37. Every check (IDs, row count, columns, positive values) passed on the written files and in Spotter's `score.py`, and a second full run reproduced both files byte for byte. There are no labels for Nov–Dec, so as a sanity check only, the predicted spread matches the training spread:

Equipment	Loads (train / predict)	Training median rate	Predicted median rate
All	48,000 / 12,000	$2,030.76	$2,029.69
Dry Van	27,202 / 6,780	$1,953.04	$1,943.21
Flatbed	8,753 / 2,169	$2,076.81	$2,090.88
Reefer	12,045 / 3,051	$2,196.67	$2,215.06

7. December prediction chart (from score.py)



For the fixed Lexington to Fort Wayne load (360 miles, Dry Van, 32,000 lb), predictions range from **$818.21 on Sundays to $837.61 on Thursdays**, in an exact weekly cycle, because only the date changes and the model's only date input is the weekday. The chart's y-axis is zoomed to the prediction range, so this $19.40 range looks larger than it is. The model has no holiday information: Christmas Day falls on a Thursday and is priced as one of the most expensive days ($837.61).

8. Limitations

- Holiday-season (Nov–Dec) price levels could not be tested, and a model without market features cannot follow future level shifts.
- The December chart is an exact weekly repeat; Christmas Day is priced like any other Thursday, the highest day of the week.
- The chart's y-axis is zoomed, so the $19.40 weekly range looks large.
- The Sep–Oct holdout was seen three times (second look pre-registered, third report only, all disclosed; no choice was made from the third).

- The weekday gain is within noise; it was kept because it passed the adoption rule.
- The bootstrap covers which test days were sampled, not a different training history or months after August.
- About 0.8% clean-row under-pricing remains on Sep–Oct.
- Laredo lies outside the area covered by the training cities; the unseen-city test only covered cities inside that area.
- The meanings of market_index and quote_signal are unknown.
- The random-vs-time and unseen-city results were measured on the Phase 4 tuned model, not the submitted model (the two differ only in boosting settings).

9. AI assistance

I used Claude (Anthropic) as a mentor throughout: it explained options and trade-offs, drafted code and the wording of this report from my logged results. I chose every option, ran every step myself, checked every output, and can explain each part of the code. Every number in this report is either printed by `src/report_numbers.py` from the saved output files or taken from the decision log (DECISIONS.md) in the repository.